Experiment-5

Aim :

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

Description :

A SQL view is a virtual table based on the result of a SELECT query. Views are useful for presenting selected data, simplifying complex queries, and providing controlled access to database information.

In this experiment, two views are created. The first view, department_salary_summary, groups employees by department and displays the number of employees, total salary, and average salary. The second view, employee_hierarchy, displays employee details together with their manager information. The updatability of a simple employee view is then tested using an UPDATE statement.

A recursive Common Table Expression (CTE) is also implemented to display the complete reporting chain of employees. The recursive CTE starts from top-level employees who do not report to another employee and repeatedly joins employees with their managers.

The following SQL uses MySQL 8.0 or later syntax because recursive CTEs require support for WITH RECURSIVE.

Source Code :

```
-- Create database objects for the experiment

CREATE TABLE department
   ( department_id INT PRIMARY
   KEY,
   department_name VARCHAR(50) NOT NULL
);

CREATE TABLE employee
   ( employee_id INT PRIMARY KEY,
   employee_name VARCHAR(100) NOT NULL,
   department_id INT,
   manager_id INT NULL,
   salary DECIMAL(10,2),
   FOREIGN KEY (department_id) REFERENCES department(department_id),
   FOREIGN KEY (manager_id) REFERENCES employee(employee_id)
);

-- Insert sample departments

INSERT INTO department VALUES
(1, 'IT'),
(2, 'HR'),
```

```
(3, 'Finance');

-- Insert sample employees

INSERT INTO employee VALUES
(101, 'Amit', 1, NULL, 90000),
(102, 'Neha', 1, 101, 65000),
(103, 'Rahul', 1, 101, 60000),
(104, 'Priya', 2, NULL, 80000),
(105, 'Karan', 2, 104, 55000),
(106, 'Riya', 3, NULL, 85000),
(107, 'Arjun', 3, 106, 50000);

-- 1. View for department salary summary

CREATE VIEW department_salary_summary AS
SELECT
   d.department_id,
   d.department_name,
   COUNT(e.employee_id) AS employee_count,
   SUM(e.salary) AS total_salary,
   AVG(e.salary) AS average_salary
FROM department d
LEFT JOIN employee e
   ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;

SELECT * FROM department_salary_summary;

-- 2. View for employee hierarchy

CREATE VIEW employee_hierarchy AS
SELECT
   e.employee_id,
   e.employee_name,
   e.department_id,
   e.manager_id,
   m.employee_name AS manager_name,
   e.salary
FROM employee e
LEFT JOIN employee m
   ON e.manager_id = m.employee_id;

SELECT * FROM employee_hierarchy;

-- 3. Test view updatability using a simple single-table view

CREATE VIEW employee_basic AS
SELECT employee_id, employee_name, salary
```

```
FROM employee;

-- Update through the view

UPDATE employee_basic
SET salary = salary + 5000
WHERE employee_id = 102;

SELECT * FROM employee_basic
WHERE employee_id = 102;

-- 4. Recursive CTE to display reporting chains

WITH RECURSIVE reporting_chain AS (
    -- Anchor member: top-level employees
    SELECT
        employee_id,
        employee_name,
        manager_id,
        0 AS level,
        CAST(employee_name AS CHAR(1000)) AS reporting_path
    FROM employee
    WHERE manager_id IS NULL

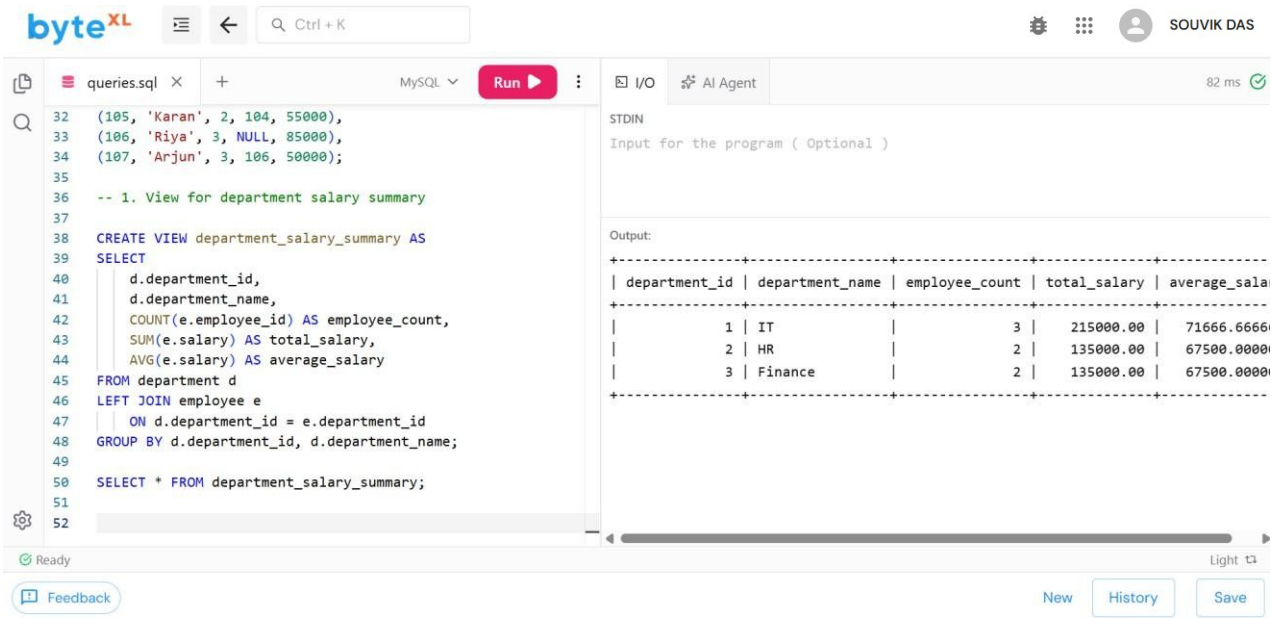
    UNION ALL

    -- Recursive member: find employees reporting to each employee
    SELECT
        e.employee_id,
        e.employee_name,
        e.manager_id,
        rc.level + 1,
        CONCAT(rc.reporting_path, ' -> ', e.employee_name)
    FROM employee e
    INNER JOIN reporting_chain rc
        ON e.manager_id = rc.employee_id
)
SELECT
    employee_id,
    employee_name,
    manager_id,
    level,
    reporting_path
FROM reporting_chain
ORDER BY reporting_path;
```

Output :

Run the SQL statements and capture screenshots showing the department salary summary, employee hierarchy, successful update through the simple view, and recursive reporting chain.

Screenshot 1: Department Salary Summary




```
(105, 'Karan', 2, 104, 55000),
(106, 'Riya', 3, NULL, 85000),
(107, 'Arjun', 3, 106, 50000);

-- 1. View for department salary summary

CREATE VIEW department_salary_summary AS
SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS employee_count,
    SUM(e.salary) AS total_salary,
    AVG(e.salary) AS average_salary
FROM department d
LEFT JOIN employee e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;

SELECT * FROM department_salary_summary;
```

```
+---------------+-----------------+---------------+--------------+-------------
| department_id | department_name | employee_count | total_salary | average_sala
+---------------+-----------------+---------------+--------------+-------------
|             1 | IT              |             3 |    215000.00 |   71666.6666
|             2 | HR              |             2 |    135000.00 |   67500.0000
|             3 | Finance         |             2 |    135000.00 |   67500.0000
+---------------+-----------------+---------------+--------------+-------------
```

Screenshot 2: Employee Hierarchy




```
INSERT INTO employee VALUES
(101, 'Amit', 1, NULL, 90000),
(102, 'Neha', 1, 101, 65000),
(103, 'Rahul', 1, 101, 60000),
(104, 'Priya', 2, NULL, 80000),
(105, 'Karan', 2, 104, 55000),
(106, 'Riya', 3, NULL, 85000),
(107, 'Arjun', 3, 106, 50000);

CREATE VIEW employee_hierarchy AS
SELECT
    e.employee_id,
    e.employee_name,
    e.department_id,
    e.manager_id,
    m.employee_name AS manager_name,
    e.salary
FROM employee e
LEFT JOIN employee m
    ON e.manager_id = m.employee_id;

SELECT * FROM employee_hierarchy;
```

```
+-------------+---------------+---------------+------------+--------------+----------+
| employee_id | employee_name | department_id | manager_id | manager_name | salary   |
+-------------+---------------+---------------+------------+--------------+----------+
|         101 | Amit          |             1 |       NULL | NULL         | 90000.00 |
|         102 | Neha          |             1 |        101 | Amit         | 65000.00 |
|         103 | Rahul         |             1 |        101 | Amit         | 60000.00 |
|         104 | Priya         |             2 |       NULL | NULL         | 80000.00 |
|         105 | Karan         |             2 |        104 | Priya        | 55000.00 |
|         106 | Riya          |             3 |       NULL | NULL         | 85000.00 |
|         107 | Arjun         |             3 |        106 | Riya         | 50000.00 |
+-------------+---------------+---------------+------------+--------------+----------+
```

Screenshot 3: View Updatability Test



Screenshot 4: Recursive CTE Reporting Chain

